BAIL security

SECURITY AUDIT

Symbiotic

Async/Account

FINAL REPORT

July 2026

BAILSEC.IO

Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Async/Account – Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	6	2		4		
Low	24	2	1	21		
Informational	22	3	1	18		
Governance						
Total	52	7	2	43		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

3.1 Scope 1

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/8fbde1bccc4400fc952f1484478209144dad8f06/src/contracts/adapters/ll-adapter/MidasAccount.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/blob/32ade1b36ccc2472668382723c39e4eb97a3cced/src/contracts/adapters/ll-adapter/MidasAccount.sol

CutoffMidasAccount

The CutoffMidasAccount contract is a Midas redemption account variant that groups pending redemption requests into monthly cutoff cohorts instead of pricing them continuously. Each request is stamped with the bucket active at submission time, and once a cohort's cutoff date passes, all requests in that bucket freeze at the last Chainlink price printed before the cutoff rather than continuing to track the live oracle rate. It combines MidasAccount, which handles the underlying redemption-vault plumbing, with CutoffAccount, which supplies the generic bucket/timestamp mixin.

Invariants

- Every entry in requestToBucket corresponds to the currentBucket() value at the time that request was submitted.
- New redemption requests are only submitted outside the owner bypass when within PRE_CUTOFF_WINDOW of the next cutoff and past COOLDOWN since the last request.

Issue_01	bucketToTimestamp month-boundary corruption when CUTOFF_DAY > 28
Severity	**Medium**
Description	In bucketToTimestamp, the starting timestamp of a bucket is computed. However, if the CUTOFF_DAY is greater than 28, then erroneous timestamps are calculated for all months that do not have CUTOFF_DAY number of days. (February for 29, multiple months for 31). Say the initial timestamp is set to 31st January 2026, 00:00. Then, for bucket = 2, bucketToTimestamp calculates as follows: *month = 1+2-2 = 1* *date_year = 2026* *date_month = 2* *date_day = 31* *date_hour = 0* But calling DateTimeLib.dateTimeToTimestamp(2026,2,31,0,0) returns 3rd March instead. Thus, there is a discrepancy and bucket boundaries overlap during the period 1-3rd March.
Recommendations	Ensure CUTOFF_DAY<28 in the constructor.
Comments / Resolution	Fixed by ensuring CUTOFF_DAY<=28 in the constructor.

Issue_02	mGLOBAL sync can revert on sub-minimum balances or Midas access-control failures, blocking finalization and swaps
Severity	Medium
Description	CutoffMidasAccount requests redemption whenever the sync path decides a request should be submitted and the account has a positive mGLOBAL balance. It does not check whether the balance satisfies Midas minimum redemption requirements before calling the Midas redemption vault. The Midas redemption request can revert when the balance is below the minimum amount, when the redemption function is paused, or when access-control checks fail. Because sync performs request finalization and request submission in the same transaction, a submission revert also rolls back any finalization that happened earlier in the same call. This also affects swaps. LiquidLaneAdapter transfers mGLOBAL into the account and then calls sync inline. If sync reverts because the resulting balance cannot be submitted to Midas, the whole swap reverts.
Recommendations	Check the Midas minimum redemption amount before attempting to submit a request. If the balance is below the minimum, skip request creation but still allow finalization of existing requests.
Comments / Resolution	Acknowledged.

Issue_03	Permissionless sync can grief real mGLOBAL redemptions
Severity	**Low**
Description	CutoffMidasAccount.sync() is is permissionless. During the final cooldown period before a monthly cutoff, anyone can send a small redeemable amount of mGLOBAL to the account and call sync(). This creates a small redemption request and updates lastRequestTimestamp. If a real swap later sends a much larger mGLOBAL amount to the account before the cutoff, the adapter's sync() call cannot create a new redemption request because the cooldown is still active. The swap still pays the user, but the larger mGLOBAL balance remains idle in the account. As a result, that larger amount can miss the current monthly redemption cohort and be delayed until the next cycle unless the owner manually calls sync() before the cutoff.
Recommendations	Consider making the sync function admin-only, or documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Round pricing discrepancy
Severity	**Low**
Description	The documentation states that "*pending requests compound until the cohort pricing date, then freeze at the first vault-feed print at/after it.*" This is inaccurate. In the current implementation, the cohort price is set at the last valid price before the end of the cohort, not the first price available after the end. This is evident from the loop shown below. *uint48 nextBucketTimestamp =* *bucketToTimestamp[requestToBucket[requestId] + 1];* *while (timestamp >= nextBucketTimestamp) {* *- -roundId... // last oracle price of the cohort*
Recommendations	Consider fixing the comment to reflect the price recording behaviour.
Comments / Resolution	Fixed by changing the comment.

Issue_05	Unbounded backward round walk can be gas-intensive
Severity	**Low**
Description	When valuing pending assets, the contract iterates over multiple chainlink rounds to find the accurate price, working backwards. *while (timestamp >= nextBucketTimestamp) {* *(, answer,, timestamp,) =* *AggregatorV3Interface(aggregator).getRoundData(--roundId);* *}* Even within a single round, this loop can walk thousands of iterations for a feed that updates frequently, and if the cohorts are long. Thus, calling this function can become very gas-intensive, since there is no upper bound for the cost.
Recommendations	Consider having the admins cache the roundIds for pricing to fix the gas-cost of the function.
Comments / Resolution	Acknowledged.

Issue_06	Historical round used without staleness/validity checks
Severity	**Low**
Description	In _pendingAssets, the contract uses the historical price from the Chainlink aggregator directly to price the tokens undergoing redemption. The issue is that the raw price is used without any checks. The price could have been unupdated for a long time, and thus could have been too stale to use as the redemption price for the cohort. Chainlink documents that a zero updatedAt means the round is incomplete and must not be used. The project's own ChainlinkPriceFeed library already treats updatedAt == 0 as invalid data. This path calls the aggregator directly and skips that check. The Midas vault oracle implements a bound check in its getDataInBase18 function, which is also ignored here.
Recommendations	Consider adding a bound check for the historical oracle price, and ensure that the price being used has updates before and after it within a staleness threshold. Also, reject rounds with updatedAt=0, indicating incomplete rounds.
Comments / Resolution	Partially fixed by adding an updatedAt check.

Issue_07	CutoffMidasAccount skips Chainlink rounds published exactly at bucket cutoff
Severity	Low
Description	CutoffMidasAccount._pendingAssets prices pending requests by walking backward from latestRoundData until it finds a round with updatedAt strictly before bucketToTimestamp(requestToBucket + 1). Monthly cutoff buckets are fixed on a calendar schedule, but Chainlink rounds are event-driven and may land at any second. Because the loop uses a strict less-than boundary, a round whose updatedAt equals the cutoff timestamp is skipped and the prior round's price is used. If Midas publishes NAV at or near the cutoff hour, the cohort locks an older rate instead of that cutoff print. The locked price therefore depends on second-level timing between oracle publication and bucket boundaries.
Recommendations	If exact-cutoff prints should count, adjust the boundary condition so rounds at the cutoff timestamp are included. Otherwise, document that cohort pricing uses the last Chainlink round strictly before the bucket cutoff, not the first at or after it, and confirm this matches Midas's intended redemption accounting.
Comments / Resolution	Acknowledged.

Issue_08	totalAssets can revert even when there is no mGLOBAL exposure
Severity	**Informational**
Description	CutoffMidasAccount.totalAssets() always reaches _pendingAssets(). Inside _pendingAssets(), the contract validates the current Midas oracle price before checking whether there are any pending requests. This means totalAssets() can revert if the current mGLOBAL oracle is stale, unavailable, or outside bounds, even when the account has no mGLOBAL balance and no pending redemption requests. This can unnecessarily break accounting reads for an account that has no exposure requiring an oracle price.
Recommendations	Only validate the oracle when there is a token balance or pending request that actually needs pricing.
Comments / Resolution	Fixed.

Issue_09	A single invalid datapoint can brick the entire contract
Severity	**Informational**
Description	In _pendingAssets, the loop iterates over multiple rounds and checks if it has a valid price. *while (timestamp >= nextBucketTimestamp) {* *(, answer,, timestamp,) =* *AggregatorV3Interface(aggregator).getRoundData(--roundId);* *}* *if (answer <= 0) {* *revert InvalidCutoffPrice();* *}* The issue is that if a single round has price = 0, it can immediately break the contract, since the execution will revert with InvalidCutoffPrice. This can happen if the walk eventually reads invalid or empty rounds.
Recommendations	Consider ignoring an invalid price and iterating more until a valid price is found, within a bound.
Comments / Resolution	Acknowledged.

Issue_10	Direct token transfers affect accounting
Severity	**Informational**
Description	CutoffMidasAccount inherits Account.totalAssets(), which counts the account's raw token balances directly. Because of this, if USDC or mGLOBAL is sent directly to the account, those tokens are immediately included in totalAssets(), even though they did not enter through the intended flow. This can affect the reported account value and may make accounting harder to reason about.
Recommendations	Document that direct token transfers to the account affect totalAssets().
Comments / Resolution	Acknowledged.

CutoffAccount

The CutoffAccount contract is a simple contract implementing the functions currentBucket and nextCutoff. currentBucket returns the bucket number corresponding to a timestamp, and nextCutoff returns the starting timestamp of the bucket that will start after the current active one. This contract is inherited by the CutoffMidasAccount contract.

No issues were identified in this contract.

3.2 Scope 2

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/5752d056ba8de336d6f0b9716aef4a7b78b8a73b/src/contracts/adapters/ThreeFAdapter.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/blob/d18d4699c6a48f1efad8d7904a6e3427cd0dfd5a/src/contracts/adapters/ThreeFAdapter.sol

ThreeFAdapter

The ThreeFAdapter contract is a VaultV2 adapter that deploys vault assets into whitelisted 3F bridge facilitator requests. It signs offers as an ERC-1271 signer through isValidSignature using the configurable offerSigner. onRequestConsumed is the whitelisted request callback that enforces per-request bounds and minimum yield, then pulls principal from the vault via allocateExact inside a transient window and approves the request. finalizeRequest permissionlessly burns the adapter's PT and YT shares in a matured request back into assets and drops the request from tracking. totalAssets values live requests, getMaxAssets reports remaining deployable capacity, and the owner sets the signer and limits. The live principal is illiquid until finalization, so _deallocate returns zero.

Appendix: Just-in-time allocation

Allocation is driven by the request rather than by the delegator. During onRequestConsumed the adapter sets the transient flag _inConsume, which switches allocatable from zero to the base implementation, calls allocateExact on the delegator to pull exactly the principal from the vault, then clears the flag and approves the calling request for the principal. Outside this window, the adapter cannot receive vault allocations at all.

Privileged Functions

- setOfferSigner
- setLimitsPerRequest

Invariants

- allocatable is zero except inside the _inConsume window opened during onRequestConsumed.
- Each live request appears exactly once in the requests array, and requestIndex maps it to its array position plus one; a zero index means the address is not a tracked request.
- requests.length never exceeds MAX_REQUESTS.
- Every accepted request satisfies minAssetsPerRequest <= principal <= maxAssetsPerRequest, and its yield scaled by YIELD_PRECISION over principal is at least minYieldPerRequest.
- pendingAssets for a request is set when the request is consumed and cleared to zero when it is finalized or when the request address is not tracked.
- Principal deployed into a live request cannot be deallocated back to the vault; deallocate only ever returns assets that are free in the adapter.

Issue_11	totalAssets jumps when canWithdraw flips
Severity	Medium
Description	When a request is funded, the contract stores its value in pendingAssets, equal to the amount funded. It values the YT tokens at 0. The same is done in the request's convertToAssets function. When the Request's loan is repaid, the convertToAssets function now evaluates the PT and YT tokens at the actual repayment rate. This causes a sudden change in the totalAssets value the moment the canWithdraw flag in the request contract is flipped. Thus, all the yield in the Adapter is accrued in a single transaction. This makes the vault susceptible to sandwich attacks. Users can frontrun the canWithdraw flip to deposit funds into the vault at a lower asset-per-share and then withdraw immediately after at a higher asset-per-share, and capture most of the yield generated.
Recommendations	Consider implementing a way to drip the yield into the contract gradually after finalization, instead of realizing the profit in a single transaction.
Comments / Resolution	Acknowledged.

Issue_12	Matured 3F request value is counted in NAV, but is unavailable for withdrawals until finalization
Severity	Low
Description	Once a 3F request becomes withdrawable, ThreeFAdapter.totalAssets values the request through the adapter's PT/YT balances and the request's convertToAssets result. At that point, the matured request value is included in vault NAV. However, the value is still not available as vault liquidity. ThreeFAdapter._deallocate always returns 0, so the delegator cannot pull matured request value back to the vault through normal deallocation. The assets only become adapter-held free assets after someone calls finalizeRequest, which burns the adapter's PT/YT position in the request. This creates a mismatch between accounting and liquidity. The vault can include matured 3F value in totalAssets, while withdrawable cannot use that value to satisfy withdrawals.
Recommendations	Consider acknowledging and documenting this behaviour, or consider adding the finalizeRequest to the deallocation flow.
Comments / Resolution	Acknowledged.

Issue_13	Loan defaults can be bypassed by frontrunning finalizeWithdraw calls
Severity	**Low**
Description	When finalizeRequest is called, the contract redeems the PT and YT tokens into the base asset. Normally, this is profitable, but in case of defaults, this can evaluate to less than pendingAssets amount of assets. Since this loss is realized in a single transaction, users can bypass this loss by frontrunning this call and withdrawing assets before this call, forcing the other holders to bear this loss instead.
Recommendations	Consider adding a delay in vault withdrawals or acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_14	getMaxAssets ignores per-request principal limits, and request length limit
Severity	Low
Description	getMaxAssets is meant to tell 3F requestors the largest principal amount that can be funded into a new request right now. It only considers delegator headroom and vault withdrawable liquidity. It does not apply maxAssetsPerRequest or minAssetsPerRequest, even though onRequestConsumed enforces both on every consume. That mismatch breaks the function's stated contract. A requestor that sizes an offer from getMaxAssets can still revert on consume for reasons the view never signals. If capacity is 50 but minAssetsPerRequest is 100, getMaxAssets returns 50, even though a request of that size is not possible. Furthermore, if the requests.length matches MAX_REQUESTS already; no further requests can be consumed, irrespective of what getMaxAssets returns.
Recommendations	Update getMaxAssets to apply the same per-request bounds used in onRequestConsumed. Cap the result at maxAssetsPerRequest. If the capped value is below minAssetsPerRequest, or if requests is already at the max limit, return 0 so callers know no valid request size is available.
Comments / Resolution	Acknowledged, documented.

Issue_15	Request finalization accepts zero or partial recovery without checks or loss reporting
Severity	Informational
Description	finalizeRequest calls burnAll on the 3F request but ignores the returned principal and yield amounts. It always clears pendingAssets, removes the request from tracking, and emits FinalizeRequest with no recovered amount. The adapter never checks whether recovery is below the original principal, whether nothing was returned at all, or whether finalization left the adapter with the expected assets. If a 3F request defaults or repays less than expected, finalization still succeeds. The vault keeps no on-chain record of how much was lost.
Recommendations	Consider documenting and acknowledging this issue, or reverting and requiring explicit handling in case of a shortfall during finalization.
Comments / Resolution	Acknowledged.

Issue_16	Zero-principal requests can consume adapter request slots
Severity	**Informational**
Description	onRequestConsumed only rejected requests where principalAssets is below minAssetsPerRequest. If minAssetsPerRequest is set to zero, a whitelisted request can consume with principalAssets = 0. The yield check is skipped when the principleAssets is zero, but the request is still added to requests and consumes one of the MAX_REQUESTS slots This can let zero-principal requests fill adapter capacity until they are finalized.
Recommendations	Rejects zero-principal requests explicitly, or require minAssetsPerRequest to be greater than zero.
Comments / Resolution	Acknowledged.

Issue_17	getMaxAssets triggers permissionless sweepPending side effects
Severity	**Informational**
Description	ThreeFAdapter.getMaxAssets looks like a capacity query but is not read-only. Every call runs UniversalDelegator.sweepPending, which processes the vault's pending withdrawal queue: sweeping free assets from adapters, filling the queue, and requesting deallocation from others as needed. Because sweepPending is permissionless, any account can trigger this sweep by calling getMaxAssets. Integrators using staticcall will revert on a state-changing call. Those using a normal call mutate vault state while only trying to read capacity.
Recommendations	Document that getMaxAssets is state-mutating and must not be called via staticcall.
Comments / Resolution	Acknowledged, documented.

Issue_18	Offer signer rotation invalidates pending offers with no grace period
Severity	**Informational**
Description	The owner can change offerSigner at any time via setOfferSigner, with no delay or overlap window. Offers must be signed by the current signer, and the 3F Request contract checks that inside consume before calling onRequestConsumed. That check and callback run in the same transaction, so offerSigner cannot change between them on-chain. The issue is rotation timing: offers signed under the old key but not yet consumed will fail once the signer is updated. When consume runs, validation uses the new signer. Old signatures revert with an invalid signature error before onRequestConsumed runs. No funds move, but the offer cannot be completed.
Recommendations	Consider timelocking setOfferSigner and clearing pending offers before rotating. Document that rotation immediately invalidates unconsumed offers signed under the previous key.
Comments / Resolution	Acknowledged, documented.

Issue_19	Stale offers fail when limits change without a transition period
Severity	**Informational**
Description	Off-chain 3F tooling sizes signed offers using getMaxAssets and the adapter's per-request limits. Both read live state: delegator headroom, vault withdrawable balance, sweep status, and current min/max/yield settings. All of these can change instantly, invalidating any in-flight signed offers instantly. The owner can call setLimitsPerRequest, the delegator admin can call setLimits, and capacity can shrink when other requests fill, liquidity drops, or a sweep is pending. There is no timelock or pending-limit state. If the pre-signed offers are used, it will lead to a revert.
Recommendations	Consider adding a timelock for changing the parameters, or managing the coordination of this off-chain.
Comments / Resolution	Acknowledged.

Issue_20	Adapter rejects compact ERC-2098 EOA signatures accepted by 3F request contracts
Severity	Informational
Description	ThreeFAdapter.isValidSignature validates offers by delegating to OpenZeppelin SignatureChecker.isValidSignatureNow(offerSigner, hash, signature). In this checkout, the OpenZeppelin EOA path calls ECDSA.tryRecover(hash, bytes signature), which only accepts 65-byte ECDSA signatures. The 3F request contracts use Solady's SignatureCheckerLib, whose EOA path accepts both standard 65-byte signatures and 64-byte ERC-2098 compact signatures. When the adapter is the offer.maker, the request validates the offer through the adapter's ERC-1271 isValidSignature function. As a result, a compact 64-byte signature can be rejected by the adapter.
Recommendations	Consider supporting ERC-2098 compact signatures in ThreeFAdapter.isValidSignature, by using the same checker as that of 3F. Alternatively, document that the 3F adapter only supports 65-byte EOA signatures.
Comments / Resolution	Acknowledged, documented.

Issue_21	Adapter cannot selectively cancel its own 3F offer nonces
Severity	**Informational**
Description	3F request contracts support offer cancellation through setNonce(newNonce). The nonce is keyed by msg.sender, meaning only the maker address can advance its own nonce. For offers where ThreeFAdapter is the maker, the nonce owner is the adapter contract itself. The adapter exposes setOfferSigner, but it does not expose any function that forwards a nonce update to a 3F request contract. This prevents selective cancellation of a specific pending offer nonce for a specific request through the adapter. Operators must rely on broader controls such as rotating offerSigner, setting the signer to zero, changing request limits, or waiting for offer expiration.
Recommendations	Add an owner-only adapter function that calls setNonce(newNonce) on a whitelisted 3F request. Validate that the target request is whitelisted and uses the vault asset. Document signer rotation as a coarse emergency cancellation path, not a replacement for request-specific nonce management.
Comments / Resolution	Fixed following recommendations.

Issue_22	setRequestNonce cannot cancel offers while request is paused or not whitelisted
Severity	Low
Description	setRequestNonce reuses _validateRequest which only accepts requests with active whitelisted status and matching asset. If a request is paused or removed from the whitelist, the adapter owner cannot advance the nonce for that request until it is enabled/whitelisted.
Recommendations	Consider allowing nonce cancellation for requests that are not enabled or whitelisted.
Comments / Resolution	Acknowledged.

3.3 Scope 3

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/42b4c2e5dc36ded4b0390534926a87e48ce40bfe/src/contracts/adapters/ll-adapter/common/AsyncRedeemAccount.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/tree/67bad5bc94cf05f5dc8139b3f1a45669a778dea1

AsyncRedeemAccount

The AsyncRedeemAccount contract is an abstract base account for integrating with ERC-7540 async redeem vaults, extending CooldownAccount with request-based redemption handling. It tracks the identifiers of its open redemption requests in the requestIds array and resolves the target async redeem vault dynamically by querying the ERC-7575 share token for the vault associated with the account asset. _totalAssets values the account as the sum of all pending redeem requests converted to assets at current prices plus the claimable amount reported by maxWithdraw at fulfillment prices.

Invariants

- Every id in requestIds has a nonzero pendingRedeemRequest in the vault after _finalizeRequests runs
- requestIds contains no duplicate request ids
- After _requestRedeem, the account's entire token-to-redeem balance has been submitted to the vault for redemption
- _totalAssets equals the asset value of all pending requests plus the currently claimable withdraw amount

Issue_23	_finalizeRequests can pop a fulfilled redemption without claiming it, stranding assets from the automated flow
Severity	Medium
Description	In _finalizeRequests, the maxRedeem function is called, and if >0, redeem is called to claim the redeemed assets. *uint256 maxRedeem = IAsyncRedeemVault(asyncRedeemVault).maxRedeem(address(this));* *if (maxRedeem > 0) {* *IAsyncRedeemVault(asyncRedeemVault).redeem(maxRedeem, address(this), address(this));* *}* After this, the redemption request is popped if pending is 0. The issue is that under certain conditions, maxRedeem can return 0. This happens when the admin of Centrifuge pauses token transfers via the hooks. maxRedeem calls _canTransfer, which can be used to disable this function. *if (!_canTransfer(vault_, user, address(0), shares)) return 0;* In that case, the request will be popped without redeeming the funds, since maxRedeem = 0. Future _finalizeRequests calls will also fail to redeem funds due to the early exit in case of an empty requests array. *uint256 length = requestIds.length;* *if (length == 0) {* *return;* *}* So if the Centrifuge vault is ever paused and then sync is called in that state, requests that have been redeemed but not claimed can get popped and then be unable to be claimed. This will persist until there are fresh funds that can be redeemed in the account, and the request ID gets added again in _requestRedeem, resulting in an availability DOS.

Recommendations	Consider allowing redemptions to process even when requestIds are empty, since in Centrifuge, all requests are fungible. Otherwise, add an admin-gated redeem function to redeem positions in case requestsId is empty but positions exist.
Comments / Resolution	Fixed by following recommendations. _finalizeRequests will attempt to redeem even if there are no requestIDs.

Issue_24	Claimable redemption assets can disappear from totalAssets during transfer restrictions
Severity	Low
Description	AsyncRedeemAccount._totalAssets calculates the value of fulfilled and claimable redemptions using maxWithdraw. *assets += IAsyncRedeemVault(asyncRedeemVault).maxWithdraw(address(this));* However, Centrifuge's maxWithdraw is not only an accounting function. It also checks whether the assets can currently be transferred. If Centrifuge pauses or restricts transfers, maxWithdraw() can return zero: *if (!_canTransfer(vault_, user, address(0), shares)) return 0;* The fulfilled redemption assets still belong to the account, but AsyncRedeemAccount temporarily reports them as having zero value. This creates an artificial decrease in totalAssets and the corresponding vault share price. Users may deposit while the claimable assets are excluded and receive more shares than they should. When transfers are enabled again, maxWithdraw includes the assets and the value suddenly returns. The newly minted shares then capture part of that recovered value. The artificial recovery may also cause performance fees to be charged again on value that was never actually lost.
Recommendations	Value claimable assets separately from maxWithdraw, so transfer restrictions do not remove them from totalAssets.
Comments / Resolution	Acknowledged.

Issue_25	Pricing inconsistency in _totalAssets
Severity	**Low**
Description	In totalAssets, the parent Account contract values the vault tokens using IOracle(ORACLE).getPrice(). *function _tokenToRedeemToAssets(uint256 amount, uint256 rate) internal view returns (uint256) {* *return amount > 0 ? amount.mulDiv(rate * _unit, TO_ASSETS_DIVISOR) : 0;* *}* The pending requests are, however, valued using convertToAssets directly. *assets += IAsyncRedeemVault(asyncRedeemVault) .convertToAssets(IAsyncRedeemVault(asyncRedeemVault).pendingRedeemRequest(requestIds[i], address(this)));* This creates an inconsistency, since the oracle price implements a min-max price gate, which is skipped when using raw convertToAssets.
Recommendations	Consider using the oracle price for both components.
Comments / Resolution	Fixed.

Issue_26	The account cannot deal with cancelled redemptions
Severity	**Low**
Description	Centrifuge vault expects users to call claimCancelRedeemRequest if part of their redemption is cancelled. The account, however, does not implement any way to do so.
Recommendations	Consider adding an admin-gated function to call claimCancelRedeemRequest.
Comments / Resolution	Acknowledged.

Issue_27	Centrifuge redeem can leave dust amounts of funds unprocessed
Severity	**Low**
Description	The Centrifuge contract natspec here warns that when using redeem, *When claiming redemption requests using redeem(), there can be some precision loss leading to dust. It is recommended to use withdraw() to claim redemption requests instead.* Thus, using redeem can lead to dust funds being left in the vault.
Recommendations	Consider following the recommendation and using withdraw instead.
Comments / Resolution	Acknowledged.

Issue_28	Fulfillment can cause sudden NAV changes
Severity	**Low**
Description	Pending requests are valued using the current convertToAssets price. After fulfillment, maxWithdraw uses the actual execution value. If these values differ, totalAssets changes suddenly. Users may deposit before an increase or withdraw before a decrease.
Recommendations	Use consistent pricing before and after fulfillment.
Comments / Resolution	Acknowledged.

Issue_29	Instant redemption swap reverts when Centrifuge requestRedeem fails
Severity	Low
Description	LiquidLaneAdapter._swap moves shares into the redeem account and then calls sync in the same transaction. Sync first finalizes claimable redemptions, then, when cooldown allows and the account holds shares, submits the full share balance via Centrifuge requestRedeem. There is no isolation: if that call reverts, the entire swap reverts and the market maker cannot complete instant redemption. That couples swap liveness to Centrifuge request-side conditions unrelated to the swap itself. In the referenced path, requestRedeem can fail on share transfer restrictions, a pending cancel, messaging/subsidy failures, or similar vault-side checks. The account owner is the curator, not the adapter, so swaps only trigger a new request on the first sync or after cooldown, but once that window is open, a failing request still aborts the swap. Idle shares are already accepted during cooldown and priced by the oracle, so failing the market-maker flow solely because a later request cannot be submitted is unnecessary coupling and an avoidable denial of the instant-redeem path when the pool or Centrifuge stack refuses or cannot process requests.
Recommendations	Isolate request submission from the swap path: catch or skip a failing requestRedeem (and optionally separate finalize from request) so sync can leave shares idle when Centrifuge will not accept a new request, without reverting _swap.
Comments / Resolution	Acknowledged.

Issue_30	getPrice is called even for a 0 amount
Severity	**Informational**
Description	In _totalAssets, the tokens are valued using the following snippet. *assets = _tokenToRedeemToAssets(amount, IOracle(ORACLE).getPrice());* The issue is that getPrice is called even when it is unnecessary, if amount = 0. This is avoided in the Account contract, since getPrice is called only if necessary (amount>0). *function _tokenToRedeemToAssets(uint256 amount) internal view virtual returns (uint256) {* *return amount > 0 ? _tokenToRedeemToAssets(amount, IOracle(ORACLE).getPrice()) : 0;* *}* This means that if the oracle is returning a faulty value, the _totalAssets function can revert even if it does not hold any tokens using that oracle.
Recommendations	Consider using the same _tokenToRedeemToAssets(uint256 amount) function as present in the Account contract.
Comments / Resolution	Fixed.

Issue_31	Cross-asset redemption proceeds are valued 1:1 after decimal normalization
Severity	Informational
Description	In AsyncRedeemAccount, when REDEMPTION_TOKEN differs from the account asset (_asset), claimable withdrawals and held redemption-token balances are converted into account-asset units only by matching decimals. No oracle or market price is used, so the contract treats one unit of the redemption token as equal to one unit of the account asset. That assumption fails whenever the tokens trade at different prices, including same-peg stablecoins during a depeg or temporary drift.
Recommendations	Price redemption-token amounts with an oracle (or other market rate) when converting to `_asset`, or restrict `REDEMPTION_TOKEN` to equal `_asset` and enforce that at deployment.
Comments / Resolution	Acknowledged.

Issue_32	Vault changes can make redemption requests inaccessible
Severity	**Informational**
Description	AsyncRedeemAccount does not store which vault created each request. Instead, it always reads the current vault: *function _asyncRedeemVault() internal view virtual returns (address) {* * return IERC7575Share(TOKEN_TO_REDEEM).vault(_asset);* *}* If Centrifuge removes or replaces that vault while a request is active, the account stops using the original vault. This can make totalAssets() and sync revert or exclude the old redemption from accounting and finalization.
Recommendations	Store the vault used for each request until the request is settled.
Comments / Resolution	Acknowledged.

Issue_33	Deallocate sweeps realized asset balance only, not claimable redemptions
Severity	Informational
Description	LiquidLaneAdapter.deallocate pulls each account's ERC-20 asset balance and sends it to the adapter for the delegator. It does not call sync or redeem first. For async redeem accounts, fulfilled redemptions sit as claimable vault value (maxWithdraw) until sync runs _finalizeRequests and redeem moves assets onto the account. Until then, deallocate and freeAssets only see the realized balanceOf(asset), while totalAssets already counts that claimable amount. Funds are not lost or stealable; they just stay claimable until a separate sync, so a deallocate that expects "all available proceeds" can return less than the accounted claimable value.
Recommendations	Document that operators should sync (or otherwise finalize claimable redemptions) before relying on deallocate/freeAssets for full proceeds, or have deallocate finalize claimable redemptions before sweeping balances.
Comments / Resolution	Acknowledged.

Issue_34	Unsupported ERC7540 operations
Severity	**Informational**
Description	The contract states that it is built to support ERC7540 vaults. However, certain operations in the contract are only applicable to Centrifuge vaults, and don't actually follow the ERC 7540 vault standard. 1. maxWithdraw in ERC7540 inherits the definition from ERC4626, representing the maximum amount that can be redeemed by the user. However, in Centrifuge and the account contract here, maxWithdraw is expected to represent the redeemed but not yet claimed amount. 2. _asyncRedeemVault defined in the contract is only applicable for ERC7575 multi-currency vaults. It is not applicable for base ERC7540 vaults.
Recommendations	Consider either documenting that this contract is for Centrifuge only, or implementing ERC7540 spec functions here and then overriding them in CentrifugeAccount.
Comments / Resolution	Partially fixed. _asyncRedeemVault now supports normal ERC7540 operations.

Issue_35	Pending redemptions accounting assumes a live request price
Severity	**Informational**
Description	ERC 7540 does not require a redemption request to remain priced at the current convertToAssets rate, AsyncRedeemAccount nevertheless values every pending request by passing its pending shares to the vault's current convertToAssets function. A compatible vault that fixes the redemption rate at submission or at another point before fulfillment can therefore be overvalued or undervalued while the request remains pending. The incorrect value would propagate into VaultV2 NAV, share pricing, fees, and allocation limits. The current Centrifuge integration is not affected because its pending requests intentionally follow the live price, and the exact asset value is established during fulfillment.
Recommendations	Document that AsyncRedeemAccount only supports vaults whose pending requests can be valued using the current convertToAssets rate.
Comments / Resolution	Acknowledged.

AsyncRedeemOracle

The AsyncRedeemOracle contract prices the shares of an ERC-7540 async redeem vault in 1e18 precision, extending the bounded Oracle base, which rejects any price outside its immutable MIN_PRICE and MAX_PRICE range. It serves as the price source for accounts in the ll-adapter system that hold the vault's share token as their token to redeem.

Invariants

- getPrice always returns a value within [MIN_PRICE, MAX_PRICE] or reverts
- The returned price is expressed in 1e18 precision regardless of the share and asset token decimals
- ASYNC_REDEEM_VAULT, SHARE_UNIT, and ASSET_UNIT are fixed at deployment and never change

Issue_36	Oracle and account vaults are not validated
Severity	**Informational**
Description	AsyncRedeemOracle permanently prices shares through its configured ASYNC_REDEEM_VAULT, while AsyncRedeemAccount resolves TOKEN_TO_REDEEM.vault(_asset). Initialization does not verify that these vaults match or that the oracle vault uses the account's token and asset. Incorrect configuration can therefore price shares through one vault while routing redemptions through another. This may produce incorrect swap quotes and cause totalAssets to change after shares enter the redemption process.
Recommendations	Validate during initialization that the oracle vault matches the account vault, asset, and share token.
Comments / Resolution	Acknowledged.

3.4 Scope 4

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/bab77c38dd883049723199e396d8b1d9ffd70f07/src/contracts/adapters/ll-adapter/FigureAccount.sol https://github.com/symbioticfi/core-mirror/blob/bab77c38dd883049723199e396d8b1d9ffd70f07/src/contracts/adapters/ll-adapter/OpenEdenAccount.sol
Resolution 1	

FigureAccount

The FigureAccount contract is a cooldown account that redeems the Figure/Hastra token-to-redeem into the underlying redemption token through the wYLDS async redeem vault. It extends CooldownAccount and plugs the Figure-specific redemption flow into the shared cooldown lifecycle. _requestRedeem unwraps any held token-to-redeem into wYLDS, transfers the wYLDS to a freshly cloned FigureSubAccount, and initializes it so the subaccount files the redemption request. _finalizeRequests sweeps redemption tokens from subaccounts whose requests have been fully processed off-chain and removes them from the subAccounts array. _totalAssets values held wYLDS, pending redemption requests, and redemption token balances across the account and all subaccounts, converting to vault assets via the oracle when the redemption token differs from the vault asset.

Invariants

- Every entry in subAccounts has a pending redemption or an unswept redemption token balance.
- A subaccount is removed only after its pending shares reach zero, and its redemption tokens are swept.

Issue_37	wYLDS tokens stranded in subaccounts on cancellation
Severity	Medium
Description	The FigureAccount contract creates subaccounts for each redemption, sends the wYLDS tokens to the subaccount, and then triggers a redemption through the subaccount. If the redemption goes through, the account then draws back the redeemed tokens. However, if the redemption is cancelled, the wYLDS tokens are then stuck in the subaccount and not valued in totalAssets. There is no viable path to recover those tokens as well, leading to a material loss
Recommendations	Consider adding a path to handle redemption cancellations, allowing the FigureAccount to recover wYLDS tokens from the subaccounts.
Comments / Resolution	Acknowledged.

Issue_38	Unclaimed wYLDS Merkle rewards in FigureAccount
Severity	**Low**
Description	Figure vault shares (wYLDS) redeem 1:1 with the underlying asset. Yield is paid separately: the Figure admin deposits rewards and publishes Merkle epochs. Eligible holders must call claimRewards with valid proof to receive newly minted wYLDS. FigureAccount holds wYLDS in two places. After instant-redeeming PRIME/AUTO, it keeps wYLDS on the main account until _requestRedeem runs. It then moves wYLDS into FigureSubAccount clones while async redemption is pending. During both periods, the account can hold wYLDS for a meaningful time. The account never calls claimRewards. Claims are tied to msg.sender, so no external party can claim on its behalf. _totalAssets only counts existing wYLDS balance, not unclaimed rewards, so missed claims are invisible to Symbiotic accounting. If Figure includes FigureAccount or its subaccounts in a reward snapshot, those rewards are never collected. Symbiotic vault depositors lose yield that should have increased account assets before final redemption to USDC.
Recommendations	Add a permissionless function on FigureAccount that accepts epoch index, amount, and Merkle proof and calls claimRewards on the wYLDS vault. Invoke it from _sync or allow keepers to call it while wYLDS is held. If subaccounts can appear in reward snapshots, add the same path there.
Comments / Resolution	Acknowledged.

Issue_39	Instant redemption swap reverts when Figure requestRedeem fails
Severity	**Low**
Description	LiquidLaneAdapter._swap transfers PRIME to the redeem account and calls sync. That call is not isolated from Figure/Hastra redemption submission. When cooldown allows it, FigureAccount._requestRedeem runs two external calls in sequence: ERC4626 redeem (PRIME → wYLDS), then FigureSubAccount.initialize, which calls requestRedeem. Neither has pre-checks or error handling. Any revert propagates through sync and aborts the entire swap. Figure can revert when the vault is paused, shares are zero, a pending redemption already exists, transfers are restricted, or any other operational gate applies. If the ERC4626 redeem succeeds but initialization fails, the transaction rolls back atomically — no wYLDS is stranded and no funds are lost, but the swap still fails until Figure accepts the request.
Recommendations	Consider decoupling request submission from the swap path. Use try/catch or skip submission when Figure would reject the call. Leave idle PRIME or wYLDS in the account until a later sync can queue it.
Comments / Resolution	Acknowledged.

Issue_40	Redemption token valued at 1:1 peg without oracle when vault asset differs
Severity	Low
Description	When the redemption token is not the Symbiotic vault's underlying asset, _redemptionTokenToAssets converts amounts using decimal scaling only. It assumes one unit of the redemption token equals one unit of the vault asset, with no price oracle. For FigureAccount, wYLDS redemptions settle as USDC. If the vault also uses USDC, this conversion is skipped, and the issue does not apply. It only matters in cross-stable setups, such as a USDT vault receiving USDC redemptions. The converted value is used in totalAssets for settled balances and pending redemption amounts. That total feeds the adapter limits and share pricing. Any depeg between the two stables misstates reported assets.
Recommendations	• Prefer deployments where REDEMPTION_TOKEN matches the vault asset. • If cross-stable support is required, value redemption-token balances with a peg oracle instead of decimal scaling alone. • Document the 1:1 assumption as an explicit deployment constraint if no oracle is added.
Comments / Resolution	Acknowledged.

Issue_41	Figure redemption subaccount list can grow without a processing bound
Severity	Low
Description	FigureAccount creates a fresh cloned subaccount whenever it has wYLDS to redeem. The new clone receives the wYLDS, submits the Figure redemption request, and is then pushed into the subAccounts array. There is no maximum length for this array. There is also no cursor or bounded processing window. Both totalAssets and _finalizeRequests scan the full list every time they run. As the number of live subaccounts grows, account valuation and sync become more expensive. In normal conditions, completed requests are removed from the array. The risk appears when requests build up faster than Figure settles them, or when many small requests are created over time. The one-day cooldown limits ordinary public growth, so this is not an instant spam issue. It is more of a backlog and liveness issue. If the array becomes large enough, routine accounting and finalization can become hard/impossible to execute within practical gas limits.
Recommendations	Consider acknowledging the issue or adding bounded redemption.
Comments / Resolution	Acknowledged.

Issue_42	One failing Figure subaccount can block the whole finalization
Severity	**Informational**
Description	FigureAccount finalizes completed Figure redemptions by walking the full subAccounts array. For each tracked subaccount, it calls pendingRedemptions. If the request is complete, it reads the redemption token balance and pulls that balance back to the parent account with safeTransferFrom. This works when every subaccount behaves normally. The problem is that one failing subaccount blocks the entire loop. If pendingRedemptions, balanceOf, or safeTransferFrom reverts for a single tracked subaccount, _finalizeRequests reverts before it can process the rest of the array. This matters because sync always finalizes existing requests before it submits a new one. A single bad completed subaccount can therefore prevent healthy completed redemptions from being swept, and it can also prevent later Figure requests from being created. Since LiquidLaneAdapter calls sync during swaps, the same condition can make new swaps for that token revert until the failing subaccount is handled. This is not about ordinary pending requests. Pending requests are skipped correctly. The issue is that there is no isolation for one subaccount that starts reverting while still tracked.
Recommendations	Consider using try-catch to avoid single points of failure.
Comments / Resolution	Acknowledged.

Issue_43	Deallocate misses settled Figure redemption tokens in sub-accounts
Severity	**Informational**
Description	When a Figure redemption completes, the redemption asset is sent to the request's sub-account, not the main FigureAccount. totalAssets includes that value because it counts redemption-token balances on sub-accounts. Those tokens are only moved to the main account when sync runs _finalizeRequests. LiquidLaneAdapter.deallocate does not call sync. It only sweeps the vault asset balance on the main account. If deallocate runs before sync, settled tokens in sub-accounts are left behind even though they are already counted in adapter accounting. This creates a gap between the reported value and what can actually be withdrawn. totalAssets and freeAssets may show proceeds as available, while deallocate returns less. Funds are not lost; anyone can call sync first. The issue mainly affects operators who expect deallocation to capture all accounted proceeds in one step.
Recommendations	Document that operators should call sync on Figure accounts before relying on deallocate or freeAssets for full proceeds. Alternatively, have deallocate finalize completed Figure redemptions before sweeping balances.
Comments / Resolution	Acknowledged.

Issue_44	Missing oracle bound check
Severity	**Low**
Description	In _totalAssets, the parent Account contract values the PRIME tokens with IOracle(ORACLE).getPrice(), which uses convertToAssets but also has max/min bound checks. However, the wYLDS tokens are valued using a raw convertToAssets call, without these bound checks. *amount += IFigureYieldVault(ASYNC_REDEEM_VAULT).convertToAssets(balance);* This creates a discrepancy in the pricing algorithm, since the 2 tokens are treated differently.
Recommendations	Consider adding bound checks when valuing wYLDS tokens as well.
Comments / Resolution	Acknowledged.

FigureOracle

The FigureOracle contract is an oracle returning the Figure/Hastra redemption price in 1e18 precision, bounded by the min and max prices enforced in the Oracle base. _getPrice converts one unit of the token-to-redeem into wYLDS via ERC4626 convertToAssets, then into the redemption token via the Figure yield vault's convertToAssets, and normalizes the result by the redemption token unit.

No issues were identified in this contract.

OpenEdenAccount

The OpenEdenAccount contract is a cooldown account that redeems OpenEden HYBOND through the HYBONDExpress queued redemption flow. It extends CooldownAccount and approves the express contract for the token-to-redeem at initialization. _requestRedeem submits the account's full HYBOND balance into the express redemption queue. _finalizeRequests is a no-op because HYBONDExpress sends the redemption asset directly to the account when the queue is processed. _totalAssets values pending redemptions, queued redemptions, and settled redemption token balances; when the queue is short, it walks the queue to use per-request net asset amounts; otherwise, it falls back to valuing the queued HYBOND amount via the oracle.

Invariants

- Queued redemptions are valued at net asset amounts (redeem amount minus fee) when the queue is walkable.

Issue_45	sync can revert on sub-minimum balances blocking finalization and swaps
Severity	**Medium**
Description	The HYBONDExpress vault implements a minimum redemption amount, below which calls to requestRedeem revert. This limit is not checked in the _requestRedeem function, which is called in the sync function, so if the account currently holds less than the minimum number of tokens, it will simply revert. This is an issue since sync is called in the swap function of the LiquidLaneAdapter, so swaps will also stop due to this minimum redemption limit.
Recommendations	Consider checking the minimum redemption limit before requesting redemptions.
Comments / Resolution	Acknowledged.

Issue_46	OpenEdenAccount cannot recover escrowed HYBOND after cancelled redemptions
Severity	Low
Description	When OpenEdenAccount queues HYBOND in Express, it is the redemption sender. If Express later cancels that request and the account is banned or de-KYC'd at that time, Express does not refund HYBOND to the account. It stores the tokens in redeemEscrowBalance[OpenEdenAccount]. To recover them, someone must call claimRedeemEscrow on Express from the account itself or from an Express operator. OpenEdenAccount has no function to do this, and its multicall cannot call Express. Symbiotic has no on-chain way to recover the escrowed HYBOND. _totalAssets also ignores escrowed HYBOND, so vault NAV drops even though the tokens still belong to the account.
Recommendations	Add a permissionless claimRedeemEscrow function on OpenEdenAccount that calls Express. Include redeemEscrowBalance(address(this)) in _totalAssets until the tokens are claimed.
Comments / Resolution	Acknowledged.

Issue_47	Instant redemption swap reverts when OpenEden requestRedeem fails
Severity	**Low**
Description	LiquidLaneAdapter._swap transfers HYBOND to the redeem account and calls sync. That call is not isolated from OpenEden request submission. When cooldown allows it, CooldownAccount._sync calls _requestRedeem, and OpenEdenAccount submits the full HYBOND balance to Express.requestRedeem with no pre-checks or error handling. Any Express revert propagates through sync and aborts the entire swap, even though the failure is unrelated to the swap itself. Express can revert when redeem is paused, KYC is invalid, balance is below redeemMinimum, or offchainShares cannot cover the full amount. The last two are worse because the account always submits 100% of its balance; it cannot queue a partial amount or skip dust below the minimum. Cooldown only delays the issue. While active, _requestRedeem is not called, so small swaps succeed, and HYBOND accumulates idle. Once the cooldown expires, the next swap tries to redeem the full balance. If that total is below redeemMinimum or exceeds offchainShares, the swap reverts even though idle HYBOND is already counted in totalAssets.
Recommendations	Consider decoupling request submission from the swap path. Use try/catch or skip requestRedeem when Express would reject it.
Comments / Resolution	Acknowledged.

Issue_48	Global redeem queue length forces oracle fallback for unrelated accounts
Severity	Low
Description	When OpenEdenAccount values final-queue redemptions in totalAssets, it first checks whether the Express contract's global redeem queue is below MAX_REDEEM_QUEUE_LENGTH (30). If it is, the account walks the full queue and sums only its own entries, using each entry's locked T+2 redeemAssetAmt minus fees. If the global queue has 30 or more entries, that walk is skipped entirely. The account's own redeemInfo balance is then added to the pending-redeem bucket and priced with the current oracle rate, even though those tokens are already in the final queue with a fixed settlement amount. The mismatch is that the branch uses the total queue size across all Express users, not how many entries belong to this account. Express only exposes a global queue (getRedeemQueueLength, getRedeemQueueInfo(index)); there is no per-account index. So one account's valuation can change because of other users' redemptions, not because of its own queue state. This mainly affects totalAssets accuracy. Deposits, withdrawals, and share pricing that rely on it can drift from true backing when the global queue is full. The vault accounting can overstate or understate an account's assets depending on how far the live oracle price sits from the locked T+2 price.
Recommendations	There is no easy fix because the Express contract does not expose a way to look up redeem info by entry ID or user address, and queue indices are positional rather than fixed per request. This risk should be clearly documented.
Comments / Resolution	Acknowledged.

Issue_49	Pending-to-final redeem transition causes discontinuous NAV
Severity	**Low**
Description	In OpenEdenAccount._totalAssets, pending and final-queue redemptions are priced differently. HYBOND still in the pending queue (pendingRedeemInfo) is always converted through _tokenToRedeemToAssets using the current oracle rate from OpenEdenOracle. After OpenEden Express runs processPendingRedeems, those tokens move into the final queue (redeemInfo) and are valued from the locked T+2 snapshot: redeemAssetAmt - feeAssetAmt from each queue entry. That means the same HYBOND balance can be marked at two different asset values across one Express state change, even though no tokens have settled into the account yet. If the oracle price has moved between request time and T+2 processing, the locked amount will differ from the live oracle estimate. When the transition happens, totalAssets can jump up or down in a single step. This only affects valuation while tokens are still inside Express. It does not steal funds directly, but any deposit, withdrawal, or share conversion that reads totalAssets around that moment can mint or burn shares against a stale or suddenly shifted NAV. The timing is not controlled by OpenEdenAccount; Express's operator decides when processPendingRedeems runs, so longer delays increase the chance of oracle drift and a larger jump.
Recommendations	Document that pending redemptions are oracle-estimated while final-queue redemptions use locked T+2 amounts, and that NAV can step-change when Express moves entries between queues. Treat totalAssets as approximate around processPendingRedeems execution, especially when the oracle price has moved since the redeem was requested.
Comments / Resolution	Acknowledged.

Issue_50	Unlimited HYBOND approval granted to upgradeable Express contract
Severity	**Informational**
Description	OpenEdenAccount grants unlimited HYBOND approval to OpenEden Express at initialization. Express needs this to pull HYBOND during requestRedeem. Express is a UUPS upgradeable contract controlled by OpenEden's UPGRADE_ROLE, not Symbiotic. The approval stays active even if Express is upgraded, so Symbiotic trusts both current Express logic and any future implementation at that address. A malicious or compromised upgrade can use the existing approval to pull all HYBOND from OpenEdenAccount at any time, without calling requestRedeem. The full account balance can be stolen.
Recommendations	Approve only the amount needed for each redemption in _requestRedeem, instead of giving unlimited approval at init. Revoke the approval after each use if possible.
Comments / Resolution	Acknowledged.

Issue_51	OpenEden HYBOND has no fallback exit path
Severity	Informational
Description	OpenEdenAccount holds HYBOND as its token-to-redeem and can only dispose of it through OpenEden Express. On sync, the account calls requestRedeem on Express with its full HYBOND balance. There is no other on-chain path to move HYBOND out. The base Account contract also blocks HYBOND in convert, reverting when HYBOND is used as input. That means CoW Swap and other converter routes cannot be used as a backup. The account has no owner rescue or emergency withdrawal function. The adapter owner can only trigger sync sooner by bypassing cooldown; that still depends on Express working. Express redemption can fail or stall for several reasons: redeem pause, failed KYC checks on the account or receiver, HYBOND ban status, or Express running out of off-chain shares. When that happens, HYBOND stays in the account. Vault accounting still counts it via the oracle price in totalAssets, but it cannot be turned into a vault asset until Express accepts the redemption and later queue processing completes. This is mostly an operational dependency risk, not a direct theft bug. But if Express stays down, the account loses KYC permanently, or Express stops working for good, HYBOND in the account has no recovery route. Vault LPs would see value on paper while real liquidity is stuck.
Recommendations	Add a fallback exit for HYBOND when Express redemption is unavailable, such as owner-controlled emergency withdrawal to a designated address, and support for Express requestDirectRedeem for when the queued path fails.
Comments / Resolution	Acknowledged.

Issue_52	Immutable redemption token desyncs from Express redeem asset updates
Severity	Informational
Description	OpenEdenAccount sets REDEMPTION_TOKEN once at deployment from Express.redeemAsset. Express can later change redeemAsset via updateRedeemAsset when all queues are empty. After that change, new redemptions settle in the new token and are sent to the account. The account still only tracks the old REDEMPTION_TOKEN in _totalAssets. Queued amounts are also valued using the old token. OpenEdenOracle has the same issue. This needs a trusted Express admin action, not an external attacker. But any account deployed before the change will miss new settled funds in totalAssets, understating NAV and mispricing shares. The new token is also not auto-converted unless handled manually.
Recommendations	Read redeemAsset from Express at runtime instead of storing it immutably, or acknowledge this issue.
Comments / Resolution	Acknowledged.

OpenEdenOracle

The OpenEdenOracle contract is an oracle returning the OpenEden HYBOND net redemption price in 1e18 precision, bounded by the min and max prices enforced in the Oracle base. _getPrice calls previewRedeem on the HYBONDExpress contract for one unit of the token-to-redeem and normalizes the returned net redemption assets (after fees) by the redemption token unit.

No issues were identified in this contract.